

Vulkan 特性

1 文档说明

本文档主要用于介绍 Vulkan 特性。一方面记录个人对 Vulkan 的学习，另一方面便于在工作和个人研究中查阅。

参考：

Vulkan 教程

2 Dynamic Uniform Buffer

2.1 背景

对于普通的 uniform buffer：

- 每个 uniform buffer 数据需要绑定一个 descriptor set
- 如果有 100 个物体，每个物体都有自己的 MVP 矩阵，那么有两种做法：
 - (1) 创建 100 个 descriptor set，每个绑定一个 uniform buffer
 - (2) 每次绘制前更新同一个 uniform buffer

这会带来明显的性能问题，所以我们引入 dynamic uniform buffer，可以做到：

- 只创建一个 descriptor set

- 在一个 uniform buffer 里存储所有物体需要的数据
- 通过 vkCmdBindDescriptorSets 的 dynamic offset 参数，在绘制时指定偏移量

2.2 优点

- (1) 减少 descriptor set 数量：1 个 descriptor set 可以服务 N 个对象
- (2) 避免频繁更新 uniform buffer：数据一次性写入 buffer，在绘制时通过 dynamic offset 指定偏移量
- (3) 提升 CPU 性能：减少 API 调用和驱动开销
- (4) 对多线程友好：每个线程处理不同偏移的数据，同时更新 buffer

2.3 使用限制

(1) dynamicOffset 必须是设备属性 *minUniformBufferOffsetAlignment* 的倍数

```

1 size_t minUboAlignment =
2     vulkanDevice->properties.limits.minUniformBufferOffsetAlignment;
3 dynamicAlignment = sizeof(glm::mat4);
4 if (minUboAlignment > 0) {
5     dynamicAlignment =
6         (dynamicAlignment + minUboAlignment - 1) & ~(minUboAlignment - 1);
7 }

```

(2) 受 maxDescriptorSetUniformBufferDynamic 限制，位于结构体 VkPhysicalDeviceLimits 中，它表示一个 descriptor set 中最多可以绑定多少个 dynamic uniform buffer。

(3) 仅适用于 uniform buffer 和 storage buffer

2.4 使用场景

- 多物体的 MVP 矩阵
- 多光源渲染
- 骨骼动画的骨骼变换矩阵

3 Push Constants

3.1 背景

对于小规模、频繁变化的数据，使用 uniform buffer 的性价比不高。Vulkan 提出 push constants，可以更高效地传递数据。

3.2 优点

- 不需要分配 descriptor set 和 uniform buffer（不用分配显存）
- 直接通过 command buffer 更新数据，降低性能开销
- 适用于高频更新
- 对应 pipeline layout，可以在多个 shader stage 共享

3.3 使用限制

(1) 最大容量限制：一般为 128 字节，可以通过 `VkPhysicalDeviceLimits::maxPushConstantsSize` 来查询

(2) 必须与 pipeline layout 匹配

(3) 字节对齐（与 uniform buffer 相同）

3.4 使用场景

- 简单材质参数（动画过程中更新）
- 状态切换（少量 bool 值或浮点数的变化）

4 Specialization Constants

4.1 背景

如果要根据不同配置（如光照数量、是否使用阴影等）使用不同着色器逻辑，一般有以下两种方法：

- 创建多个着色器
- 使用 `#define`，然后多次编译

但是这会导致：

- 需要创建大量着色器
- 编译管理复杂
- 无法在运行时灵活切换

这种情况下可以考虑使用 Specialization Constants，可以实现：

- 只创建一个着色器
- 在创建管线时对常量进行动态“特化”
- 驱动/Vulkan 会进行优化（如死代码消除）

其本质是编译期常量，在管线创建时确定，但是由 CPU 在运行时提供。

4.2 优点

- 减少着色器数量：一个着色器支持不同配置
- 驱动级优化：GPU 驱动可根据常量值进行死代码消除、循环展开、分支预测等优化
- 运行时灵活：可在应用运行时根据设置创建不同特化的管线
- 无运行时开销：数值在管线创建时固化，着色器无须读取

4.3 使用限制

(1) 只在管线创建时设置，一旦管线创建完成，Specialization Constants 的值便固定了，无法在绘制时更改

(2) 支持标量和向量类型 (bool, int, vec2, vec3)，不支持结构体、数组

(3) SPIR-V 必须保留特化信息

(4) 调试难度大，一个着色器对应多个变体，需注意对应关系

4.4 使用场景

- 功能开关，如是否使用法线贴图、SSAO、阴影等
- 配置参数，如最大光源数、粒子数量、纹理采样次数等
- 平台适配，如桌面端和移动端使用不同精度数值或不同逻辑
- 调试模式，如查看法线，查看 GI 等

5 Subpass 和多个 Render Pass 的区别

5.1 内存访问模型不同

(1) Subpass:

- 在 Tile-Based GPU (如 Apple Silicon、ARM Mali、Adreno) 中，所有 subpass 的中间结果 (如 GBuffer) 都在高速片上缓存 (tile memory) 中，无需写入主显存
- Subpass 2 通过 subpassLoad() 读取 Subpass 1 写入的数据，无需写入主显存
- 仅限当前像素: fragment shader 中只能读取与当前 fragment 相同 (x, y) 位置的数据
- 无法被 Compute Shader、其它 Render Pass 或 CPU 访问

```

1 // fragment shader in Subpass 2
2 layout(input_attachment_index = 0, set = 0, binding = 0) uniform subpassInput inputAlbedo;
3 void main() {
4     vec3 albedo = subpassLoad(inputAlbedo); // 读取同一像素的 albedo
5 }

```

(2) 多个 Render Pass

- 每个 Pass 结束时，附件必须写回主显存
- 下一个 Pass 开始时，从主显存加载数据
- 可使用 `sample2D` 对任意位置进行采样

5.2 性能差异

(1) Tile-based GPU（移动端设备）：

- **Subpass**：数据始终在 tile memory 上缓存，节省大量带宽
- **多个 Render Pass**：每个 Pass 都要读写主显存，带宽开销大

(2) Desktop GPU（NVIDIA/AMD）：

- 没有 tile memory，但驱动可能将 Subpass 优化为单次渲染调度，减少状态切换

5.3 同步机制不同

(1) Subpass 间同步：

- 通过 `VkSubpassDependency` 声明执行顺序和内存依赖
- GPU 硬件自动处理，无需 CPU 干预，同步开销较低

(2) 多个 Render Pass 间同步：

- 使用 Semaphore（不同 Queue 之间）或 Pipeline Barrier（同一 Queue 内）进行同步
- 增加 CPU 开销，可能导致 GPU 空闲等待

5.4 图像布局管理不同

(1) Subpass:

- 自动处理所有附件的布局转换 (initialLayout -> subpass layout -> finalLayout)
- 开发者无需关心中间布局

(2) 多个 Render Pass:

- 必须手动管理图像布局
- 布局错误会导致未定义行为或崩溃

5.5 使用场景

场景	推荐方案
移动端延迟渲染	Subpass (利用 tile memory)
后处理 (如 <i>Bloom</i> → <i>Tonemapping</i>)	Subpass
需要 Compute Shader 处理中间结果 (SSAO)	多个 Render Pass
CPU 读取中间结果	多个 Render Pass

6 Callable Shader

6.1 定义

Callable Shader (可调用着色器) 是 Vulkan Ray Tracing (光线追踪) 的扩展引入的一种着色器类型, 用于在光追管线中实现灵活的、可嵌套的函数调用机制。

它不用于光线和几何体的相交, 而是通过 `executeCallableEXT()` 调用, 用于封装可复用的着色逻辑 (如材质计算、BRDF 计算、随机数生成等), 让其他着色器 (如 `Closet Hit` 或 `Miss`) 调用。

6.2 特点

特性	说明
调用方式	通过 <code>executeCallableEXT()</code> 从其他着色器中调用
无光线交互	不处理光线和物体求交，仅执行通用计算
共享数据	通过 Shader Binding Table (SBT) 传递参数 (如材质 ID、纹理索引等)
递归支持	可嵌套调用其它 Callable Shader (受管线最大递归深度限制)
性能优势	避免在多个 Hit Shader 中重复编写相同逻辑，提升代码复用性和维护性

6.3 应用场景

- 材质计算：不同材质对应不同 Callable Shader，Hit Shader 根据材质 ID 选择对应的 Callable Shader
- BRDF/BSDF 计算：不同光照模型对应不同 Callable Shader，避免重复实现
- 随机数生成：提供统一的随机采样逻辑

7 主命令缓冲区和次命令缓冲区

7.1 核心区别

在 Vulkan 中，命令缓冲区 (Command Buffer) 分为两种：

- 主命令缓冲区: `VK_COMMAND_BUFFER_LEVEL_PRIMARY`
- 次命令缓冲区: `VK_COMMAND_BUFFER_LEVEL_SECONDARY`

特性	PRIMARY	SECONDARY
能否直接提交到队列	可以	不可以
能否调用 <code>vkCmdDraw/vkCmdDispatch</code>	可以	特定情况下可以
能否调用 <code>vkCmdExecuteCommands</code>	可以 (执行 SECONDARY)	不可以
能否继承渲染通道状态	不可以	通过 <code>VkCommandBufferInheritanceInfo</code> 继承
用途	主渲染逻辑、计算任务、帧提交	模块化渲染 (如 UI、粒子系统)、多线程录制

7.2 具体行为

7.2.1 Primary Command Buffer

这是唯一能够通过 `vkQueueSubmit` 提交到队列的命令缓冲区，其功能包括：

- 渲染命令 (`vkCmdBeginRenderPass`, `vkCmdDraw`, `vkEndRenderPass` 等)
- 计算命令 (`vkCmdDispatch`, `vkCmdDispatchIndirect` 等)
- 资源屏障 (`vkCmdPipelineBarrier`)
- 执行 Secondary Command Buffer (`vkCmdExecuteCommands`)

7.2.2 Secondary Command Buffer

(1) 不能直接提交，必须通过 Primary Command Buffer 调用 `vkCmdExecuteCommands`

(2) 不能调用 `vkCmdBeginRenderPass`, `vkEndRenderPass`, `vkCmdExecuteCommands`

(3) 可以调用 `vkCmdDraw/vkCmdDispatch`，需要满足：

- 如果要在渲染通道内，必须提供 `VkCommandBufferInheritanceInfo`，并指定 `renderPass` 和 `subpass` 以及 `framebuffer`
- 如果不在渲染通道内（纯计算），则无需继承信息，但不能包含渲染命令

7.3 Secondary Command Buffer 的优点

(1) 多线程录制

- 不同线程并行录制多个 Secondary Command Buffer

- 主线程最后用 Primary Command Buffer 执行所有 Secondary Command Buffer
- 避免线程间同步开销（Primary 录制时是单线程的）

（2）模块化与复用

- 将固定渲染逻辑封装为 Secondary Command Buffer
- 多帧复用

（3）减少 CPU 开销

- 将复杂场景拆分成多个 Secondary Command Buffer，仅更新变化的部分

8 GPU Instancing

8.1 定义

GPU Instancing（GPU 实例化）是一项高效渲染大量相似物体的技术。它允许 GPU 仅用一次绘制调用（draw call），渲染大量具有相同网格但是不同位置/材质参数的物体，从而大幅降低 CPU 开销、提升渲染性能。

8.2 优势

传统渲染：

- CPU 瓶颈：如果渲染 N 个物体，则需要 N 个 draw call，驱动开销较大
- Uniform 更新频繁：CPU-GPU 数据传输成为瓶颈
- GPU 利用率低：GPU 等待 CPU 提交命令，无法满载

GPU 实例化：

- 1 次 draw call 渲染 N 个物体
- 实例数据由 GPU 直接获取（通过顶点属性或 SSBO）
- 降低 CPU 负载

8.3 Vulkan 实现方式

8.3.1 实例化顶点属性

将实例数据作为**第二组顶点数据**，Vulkan 自动为每个实例递增 instanceIndex。

(1) 设置实例数据，创建对应 buffer

```
1 struct InstanceData {
2     glm::vec3 pos;
3     glm::vec3 rot;
4     float scale{ 0.0f };
5     uint32_t texIndex{ 0 };
6 };
```

(2) 设置顶点输入绑定

```
1 std::vector<VkVertexInputBindingDescription> bindingDescriptions = {
2     // Binding point 0: Mesh vertex layout description at per-vertex rate
3     vks::initializers::vertexInputBindingDescription(0, sizeof(vkglTF::Vertex), VK_VERTEX_INPUT_RATE_VERTEX),
4     // Binding point 1: Instanced data at per-instance rate
5     vks::initializers::vertexInputBindingDescription(1, sizeof(InstanceData), VK_VERTEX_INPUT_RATE_INSTANCE),
6 };
```

(3) 顶点属性描述

```
1 std::vector<VkVertexInputAttributeDescription> attributeDescriptions = {
2     // Per-vertex attributes
3     // These are advanced for each vertex fetched by the vertex shader
4     vks::initializers::vertexInputAttributeDescription(0, 0, VK_FORMAT_R32G32B32_SFLOAT, 0),
5     vks::initializers::vertexInputAttributeDescription(0, 1, VK_FORMAT_R32G32B32_SFLOAT, sizeof(float) * 3),
6     vks::initializers::vertexInputAttributeDescription(0, 2, VK_FORMAT_R32G32_SFLOAT, sizeof(float) * 6),
7     vks::initializers::vertexInputAttributeDescription(0, 3, VK_FORMAT_R32G32B32_SFLOAT, sizeof(float) * 8),
8     // Per-Instance attributes
9     // These are advanced for each instance rendered
10    vks::initializers::vertexInputAttributeDescription(1, 4, VK_FORMAT_R32G32B32_SFLOAT, 0),
11    vks::initializers::vertexInputAttributeDescription(1, 5, VK_FORMAT_R32G32B32_SFLOAT, sizeof(float) * 3),
12    vks::initializers::vertexInputAttributeDescription(1, 6, VK_FORMAT_R32_SFLOAT, sizeof(float) * 6),
13    vks::initializers::vertexInputAttributeDescription(1, 7, VK_FORMAT_R32_SINT, sizeof(float) * 7),
14 };
```

(4) 绑定并绘制

```
1 // Binding point 0 : Mesh vertex buffer
2 vkCmdBindVertexBuffers(cmdBuffer, 0, 1, &models.rock.vertices.buffer, offsets);
```

```

3 // Binding point 1 : Instance data buffer
4 vkCmdBindVertexBuffers(cmdBuffer, 1, 1, &instanceBuffer.buffer, offsets);
5 // Bind index buffer
6 vkCmdBindIndexBuffer(cmdBuffer, models.rock.indices.buffer, 0, VK_INDEX_TYPE_UINT32);

```

(5) 顶点着色器

```

1 // Vertex attributes
2 layout (location = 0) in vec3 inPos;
3 layout (location = 1) in vec3 inNormal;
4 layout (location = 2) in vec2 inUV;
5 layout (location = 3) in vec3 inColor;
6
7 // Instanced attributes
8 layout (location = 4) in vec3 instancePos;
9 layout (location = 5) in vec3 instanceRot;
10 layout (location = 6) in float instanceScale;
11 layout (location = 7) in int instanceTexIndex;

```

8.3.2 实例化 SSBO

适用于实例数据较大或者动态更新频繁的场景。

```

1 // 顶点着色器
2 layout(std430, binding = 1) readonly restrict buffer InstanceBuffer {
3     InstanceData instances[];
4 };
5
6 void main() {
7     uint id = gl_InstanceIndex;
8     mat4 model = instances[id].model;
9     gl_Position = viewProjection * model * vec4(inPos, 1.0);
10 }

```

8.4 适用场景

- 草地、森林、城市、人群
- 粒子系统

9 Indirect Draw

9.1 介绍

Indirect Draw (间接绘制) 是现代图形 API (Vulkan、Direct3D 12、Metal) 中一项革命性技术, 它允许 GPU 直接从显存中读取绘制指令, 从而实现

GPU-Driven Rendering (GPU 驱动渲染)。这项技术是突破 CPU 瓶颈、实现超大规模渲染的必要手段。

9.2 和传统渲染的区别

传统渲染的瓶颈

- CPU 成为瓶颈：每帧大量 draw call，驱动开销大
- 无法利用 GPU 的并行计算能力：剔除、LOD 等操作在 CPU 串行执行
- 延迟高：GPU 利用率低

Indirect Draw

- 绘制指令存储在显存中，GPU 直接从显存中读取绘制指令并执行
- 支持 GPU 剔除
- 大幅减少 CPU 负载和驱动开销

9.3 Vulkan API 示例

命令	对应结构体
vkCmdDrawIndirect	VkDrawIndirectCommand
vkCmdDrawIndexedIndirect	VkDrawIndexedIndirectCommand

```
1 // 绘制指令结构体
2 struct VkDrawIndirectCommand {
3     uint32_t indexCount;
4     uint32_t instanceCount;
5     uint32_t firstIndex;
6     int32_t vertexOffset;
7     uint32_t firstInstance;
8 };
9
10 struct VkDrawIndexedIndirectCommand {
11     uint32_t indexCount;
12     uint32_t instanceCount;
```

```

13     uint32_t firstIndex;
14     int32_t vertexOffset;
15     uint32_t firstInstance;
16 };

```

对应的 buffer 创建必须有 `VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT`。

9.4 Vulkan 相关拓展

9.4.1 `VK_KHR_draw_indirect_count`

核心功能：

允许间接绘制命令的数量（draw count）由 GPU 缓冲区动态指定，而不是 CPU 在录制命令时指定。

解决的问题：

在标准 `vkCmdDrawIndirect` 中，`drawCount` 参数必须由 CPU 提供，但在 GPU 剔除（如视锥剔除）后，CPU 无法提前知道可见物体数量，不知道如何设置 `drawCount`。

Vulkan API

```

1  void vkCmdDrawIndirectCount(
2      VkCommandBuffer          commandBuffer,
3      VkBuffer                 buffer,
4      VkDeviceSize             offset,
5      VkBuffer                 countBuffer,
6      VkDeviceSize             countBufferOffset,
7      uint32_t                 maxDrawCount,
8  );
9
10 void vkCmdDrawIndexedIndirectCount(
11     VkCommandBuffer          commandBuffer,
12     VkBuffer                 buffer,
13     VkDeviceSize             offset,
14     VkBuffer                 countBuffer,
15     VkDeviceSize             countBufferOffset,
16     uint32_t                 maxDrawCount,
17 )

```

使用场景：

- GPU 剔除后，将可见实例数写入 `countBuffer`；
- 渲染时无需 CPU 干预，GPU 自动读取 `countBuffer` 决定绘制的命令数。

兼容性:

- 大多数支持 Vulkan1.2+ 的设备都支持（或通过扩展支持）；
- Vulkan1.2 已将其纳入核心规范（无需显式启用拓展）。

9.4.2 *VK_EXT_multi_draw_indirect*

核心功能:

允许单次 API 调用执行多个间接绘制命令（即 drawCount > 1）。

Vulkan 示例

```
1  if (vulkanDevice->features.multiDrawIndirect)
2  {
3      vkCmdDrawIndexedIndirect(cmdBuffer,
4      indirectCommandsBuffer.buffer, 0, indirectDrawCount, sizeof(VkDrawIndexedIndirectCommand));
5  }
6  else
7  {
8      // If multi draw is not available, we must issue separate draw commands
9      for (auto j = 0; j < indirectCommands.size(); j++)
10     {
11         vkCmdDrawIndexedIndirect(cmdBuffer,
12         indirectCommandsBuffer.buffer, j * sizeof(VkDrawIndexedIndirectCommand), 1, sizeof(VkDrawIndexedIndirectCommand));
13     }
14 }
```

兼容性:

- Vulkan1.1+ 已将其纳入核心规范；
- 几乎所有现代 GPU 都支持（包括移动平台）。

注意:

- 此拓展不解决 drawCount 动态问题，仍需 CPU 提供 drawCount 参数
- 通常与 *VK_KHR_draw_indirect_count* 一起使用

9.4.3 *VK_NV_device_generated_commands*

核心功能：

允许 GPU 完全生成并提交绘制命令，包括：

- 动态选择管线（pipeline）
- 动态绑定描述符集（descriptor set）
- 动态设置常量（push constants）
- 生成完整的 vkCmdDraw* 命令

解决的问题

标准 Indirect Draw 只能改变绘制参数（如 instanceCount），但不能改变管线状态、着色器、材质等，而真实场景中，不同物体可能使用不同材质/着色器。

Vulkan API

调用	作用
VkIndirectCommandsLayoutNV	定义 GPU 生成的命令结构
VkIndirectCommandsTokenNV	描述命令中每个字段
vkCmdExecuteGeneratedCommandsNV	执行 GPU 生成的命令

使用场景

- 材质多样性场景：草地（不同颜色/纹理）、城市（不同建筑材料）
- GPU 驱动的渲染管线：在 GPU 完成从剔除，到材质渲染，最后绘制的流程

兼容性

- 仅支持 NVIDIA 的 GPU
- 不支持跨平台

9.5 GPU-Driven Rendering 示例

(1) CPU 上传数据:

- 所有对象的包围盒 (AABB)
- 模型变换矩阵
- 静态网格数据

(2) GPU 计算剔除结果 (Compute Shader):

- 输入: AABB+ 相机视锥
- 输出: 更新 indirectBuffer 中的 instanceCount (可见 =1, 不可见 =0)

(3) 绘制:

- 使用 vkCmdDrawIndexedIndirectCount 绘制所有 instanceCount > 0 的对象

10 Query Pool

10.1 介绍

Query Pool 是 Vulkan 中用于从 GPU 异步获取运行时信息的核心机制。它允许应用程序在不阻塞渲染管线的情况下, 查询诸如时间戳 (Timestamp)、遮挡 (Occlusion)、管线统计 (Pipeline Statistics) 等数据。广泛应用于性能分析、GPU 剖析、可见性优化等场景。

10.2 支持的类型

此处介绍常用的三种功能, 更多功能看官方文档。

10.2.1 *VK_QUERY_TYPE_OCCLUSION*

应用场景：判断物体是否被遮挡（GPU 剔除）

```
1 vkCmdBeginQuery(cmdBuffer, queryPool, i, 0);
2 vkCmdDrawIndexed(cmdBuffer, ...);
3 vkCmdEndQuery(cmdBuffer, queryPool, i);
4
5 // 获取结果
6 vkGetQueryPoolResults(
7     device,
8     queryPool,
9     0,
10    2,
11    sizeof(passedSamples),
12    passedSamples,
13    sizeof(uint64_t),
14    VK_QUERY_RESULT_64_BIT | VK_QUERY_RESULT_WAIT_BIT);
15
16 if (passedSamples > 0) {
17     // 物体可见
18 }
```

10.2.2 *VK_QUERY_TYPE_TIMESTAMP*

应用场景：测量 GPU 执行某段命令的实际耗时（单位：纳秒）

```
1 vkCmdWriteTimestamp(cmdBuffer, VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT, queryPool, 0);
2 vkCmdDrawIndexed(cmdBuffer, ...);
3 vkCmdWriteTimestamp(cmdBuffer, VK_PIPELINE_STAGE_BOTTOM_OF_PIPE_BIT, queryPool, 1);
4
5 uint64_t timestamps[2];
6 vkGetQueryPoolResults(
7     device,
8     queryPool,
9     0,
10    2,
11    sizeof(timestamps),
12    timestamps,
13    sizeof(uint64_t),
14    VK_QUERY_RESULT_64_BIT | VK_QUERY_RESULT_WAIT_BIT);
15
16 // 获取时间戳周期（用于转换为秒）
17 VkPhysicalDeviceProperties properties;
18 vkGetPhysicalDeviceProperties(physicalDevice, &properties);
19 float timestampPeriod = properties.limits.timestampPeriod; // ns per tick
20
21 // 计算耗时
22 uint64_t delta = (timestamps[1] - timestamps[0]) * timestampPeriod / 1e6;
```

10.2.3 *VK_QUERY_TYPE_PIPELINE_STATISTICS*

应用场景：获取 GPU 处理的原始数据量

```
1 // 在创建 query pool 时指定查询
2 VkQueryPoolCreateInfo queryPoolInfo = {};
3 queryPoolInfo.sType = VK_STRUCTURE_TYPE_QUERY_POOL_CREATE_INFO;
4 // This query pool will store pipeline statistics
5 queryPoolInfo.queryType = VK_QUERY_TYPE_PIPELINE_STATISTICS;
6 // Pipeline counters to be returned for this pool
7 queryPoolInfo.pipelineStatistics =
8     VK_QUERY_PIPELINE_STATISTIC_INPUT_ASSEMBLY_VERTICES_BIT |
9     VK_QUERY_PIPELINE_STATISTIC_INPUT_ASSEMBLY_PRIMITIVES_BIT |
10    VK_QUERY_PIPELINE_STATISTIC_VERTEX_SHADER_INVOCATIONS_BIT |
11    VK_QUERY_PIPELINE_STATISTIC_CLIPPING_INVOCATIONS_BIT |
12    VK_QUERY_PIPELINE_STATISTIC_CLIPPING_PRIMITIVES_BIT |
13    VK_QUERY_PIPELINE_STATISTIC_FRAGMENT_SHADER_INVOCATIONS_BIT;
14
15 if (deviceFeatures.tessellationShader) {
16     queryPoolInfo.pipelineStatistics |=
17         VK_QUERY_PIPELINE_STATISTIC_TESSELLATION_CONTROL_SHADER_PATCHES_BIT |
18         VK_QUERY_PIPELINE_STATISTIC_TESSELLATION_EVALUATION_SHADER_INVOCATIONS_BIT;
19 }
```

返回结果包含：

- 输入顶点数
- 输入图元数
- 裁剪调用次数
- 裁剪图元数
- 顶点着色器调用次数
- 片元着色器调用次数
- 细分着色器调用次数

10.3 特性

10.3.1 异步性

- 查询结果不会立刻可用，需等待 GPU 完成；
- 使用 *VK_QUERY_RESULT_WAIT_BIT* 会阻塞 CPU，不推荐每帧使用；

10.3.2 query pool 是环形缓冲区

- queryCount 定义了池的大小
- 可重复使用槽位（如每帧使用不同索引、循环覆盖）

10.3.3 内存布局

- 结果以数组形式返回，每个查询占 stride 字节
- 支持 32 位或 64 位结果

11 Variable Rate Shading

11.1 介绍

Variable Rate Shading (VRS, 可变速率着色) 是一种现代 GPU 硬件加速技术, 允许在单个渲染 pass 中, 对屏幕不同区域使用不同的着色速率, 从而在保持画面质量的同时显著提升性能。

- **传统渲染**: 每个像素 (或每个 sample) 都执行一次 fragment shader。
- **VRS**: 允许将多个像素“合并”成一个着色调用 (如 2×2 、 4×4 像素共享一个颜色), 适用于细节不敏感区域 (如天空、背景、运动模糊区域)。

11.2 常见着色速率格式

- 1×1
- 1×2
- 2×1
- 2×2
- 2×4

- 4×2

- 4×4

11.3 工作方式

11.3.1 设置方式

主要分为附件控制、管线控制和图元控制三种策略，通过以下方式进行设置。

```
1 VkPhysicalDeviceFragmentShadingRateFeaturesKHR enabledPhysicalDeviceShadingRateImageFeaturesKHR{};
2 enabledPhysicalDeviceShadingRateImageFeaturesKHR.sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FRAGMENT_SHADING_RATE_FEATURES_KHR;
3 enabledPhysicalDeviceShadingRateImageFeaturesKHR.attachmentFragmentShadingRate = VK_TRUE;
4 enabledPhysicalDeviceShadingRateImageFeaturesKHR.pipelineFragmentShadingRate = VK_TRUE;
5 enabledPhysicalDeviceShadingRateImageFeaturesKHR.primitiveFragmentShadingRate = VK_TRUE;
6
7 // 将enabledPhysicalDeviceShadingRateImageFeaturesKHR传给device创建时的pNext参数
```

三种方式可以进行组合，组合操作支持以下类型：

```
1 typedef enum VkFragmentShadingRateCombinerOpKHR {
2     VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR = 0,
3     VK_FRAGMENT_SHADING_RATE_COMBINER_OP_REPLACE_KHR = 1,
4     VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MIN_KHR = 2,
5     VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MAX_KHR = 3,
6     VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MUL_KHR = 4,
7     VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MAX_ENUM_KHR = 0x7FFFFFFF
8 } VkFragmentShadingRateCombinerOpKHR;
```

Combine 操作	结果
<code>VK_FRAGMENT_SHADING_RATE_COMBINER_OP_KEEP_KHR</code>	$\text{Combine}(A, B) = A$
<code>VK_FRAGMENT_SHADING_RATE_COMBINER_OP_REPLACE_KHR</code>	$\text{Combine}(A, B) = B$
<code>VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MIN_KHR</code>	$\text{Combine}(A, B) = \min(A, B)$
<code>VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MAX_KHR</code>	$\text{Combine}(A, B) = \max(A, B)$
<code>VK_FRAGMENT_SHADING_RATE_COMBINER_OP_MUL_KHR</code>	$\text{Combine}(A, B) = A * B$

在实际应用，会通过指定两个 combine 运算来组合不同形式的 shading rate，得到最终的 shading rate，如图1所示。

11.3.2 附件控制（主要方式）

通过一张 shading rate image 为屏幕空间的不同区域指定不同着色速率。

```
1 VkFragmentShadingRateAttachmentInfoKHR fragmentShadingRateAttachmentInfo{};
2 fragmentShadingRateAttachmentInfo.pFragmentShadingRateAttachment = &fragmentShadingRateReference;
```

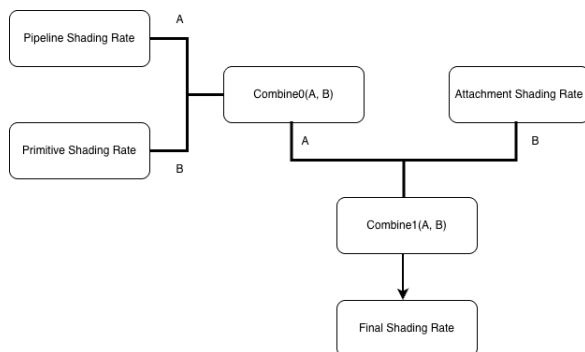


图 1: 多种形式决定最终着色速率

11.3.3 管线控制

在创建管线时使用固定着色速率，在管线创建时，填充以下结构体，并将其传递给 `VkGraphicsPipelineCreateInfo` 的 `pNext` 参数。

```

1 typedef struct VkPipelineFragmentShadingRateStateCreateInfoKHR {
2     VkStructureType sType;
3     const void* pNext;
4     VkExtent2D fragmentSize;
5     VkFragmentShadingRateCombinerOpKHR combinerOps[2];
6 } VkPipelineFragmentShadingRateStateCreateInfoKHR;
  
```

11.3.4 图元控制

在 Vertex Shader 中通过变量 `gl_PrimitiveShadingRateEXT` 设置图元的着色速率。